Frame extraction flow

This document shows how Thumb Buddy extracts frames from a video at regular intervals and converts each one to an RGBA image. To illustrate the steps, the orange notes follow one request all the way through:

- Test clip: `tests/fixtures/fixture-8bit.mp4`
- Requested time: 0.5 s
- Frame rate: 25 fps
- Resolution: 320×240

Terms used in the diagram

Packet and frame

A video file stores compressed *packets*. The decoder turns packets into *frames*, which are complete images in YUV color.

Ticks

The integer clock a video stream uses for timestamps. In the test clip, one tick is 1/12800 s, so 0.5 s is 6400 ticks.

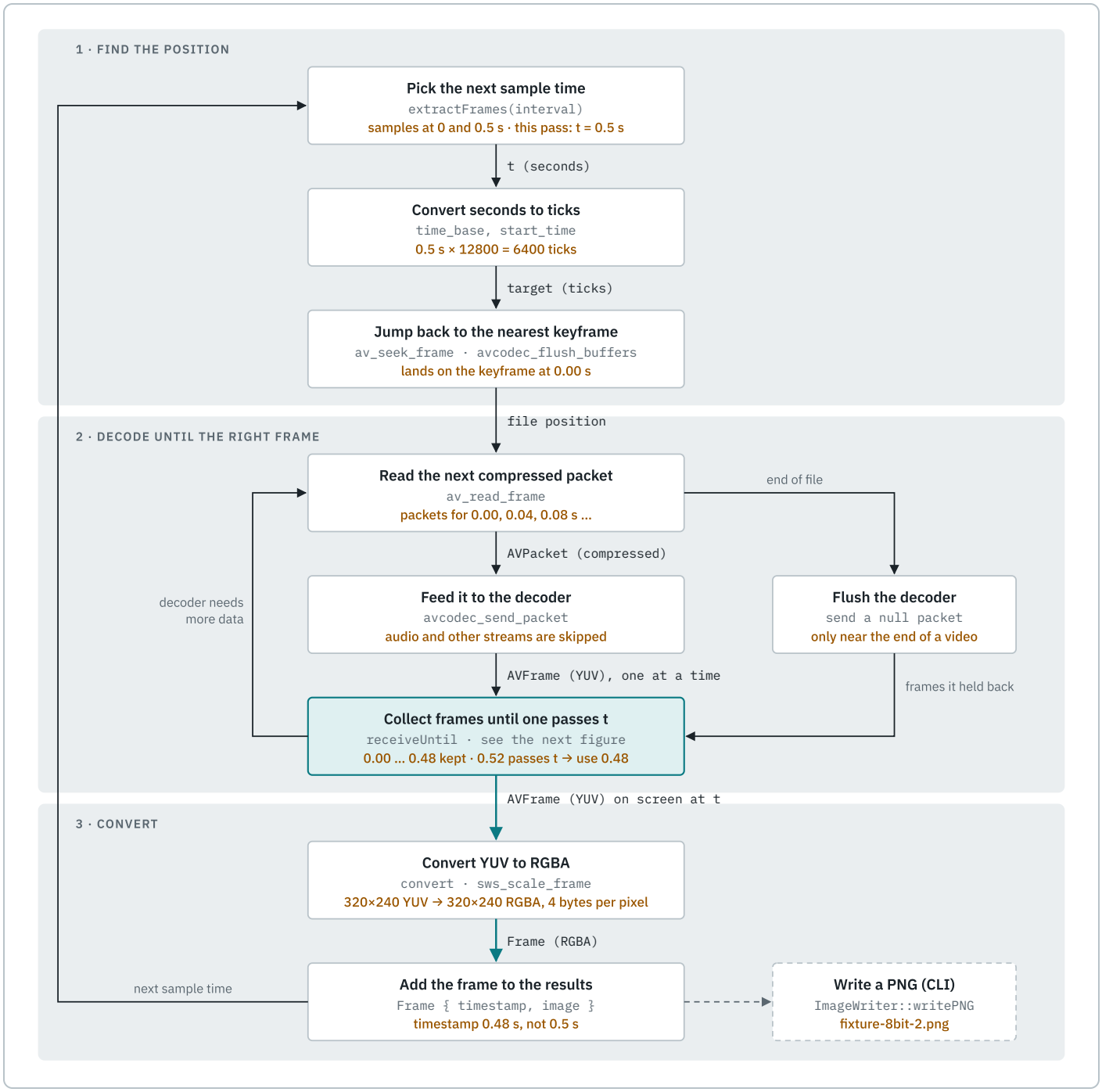
Keyframe

A frame that can be decoded on its own. Other frames are stored as changes from earlier frames, so decoding has to start at a keyframe and work forward.

Flushing the decoder

A decoder holds back a few frames while it waits for more input. At the end of the file, you have to tell it there is no more input. It then returns the frames it was holding.

One sample time, start to finish

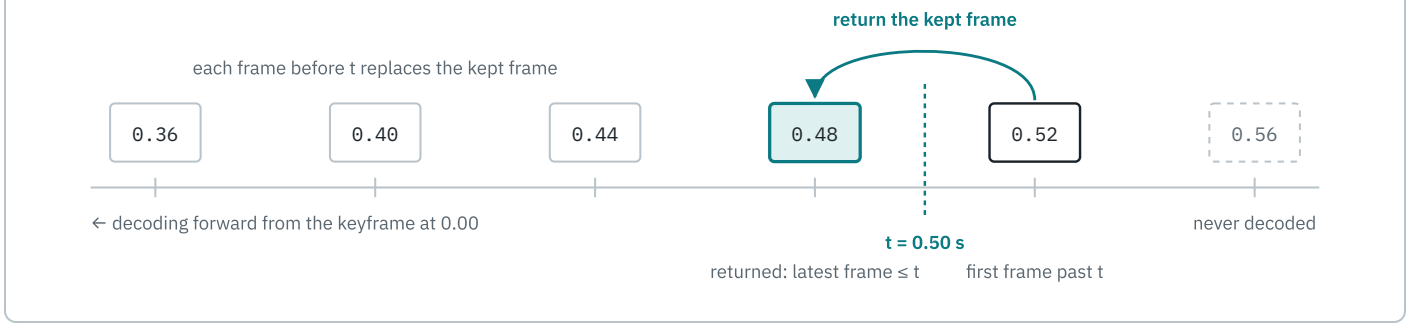


Path taken once the right frame is found Outside FrameExtractor

0.48 Worked example: 0.5 s in the test clip

Each sample time gets its own seek. This means the work per frame depends on the distance between keyframes, not on the length of the video. Any step can fail. A failure returns no frame and an error message. Failure paths are left out of the diagram to keep it readable.

Which frame is returned



At 25 fps, frames are 0.04 s apart, so there is no frame at exactly 0.50 s. At that moment, a video player is still showing the frame at 0.48 s. Requirement F5 (return the frame on screen at t) says to return that frame. Two edge cases follow the same rule: a frame exactly at t is returned as is, and a t before the first frame returns the first frame. In the code, the kept frame is `lastFrame`.

Details not shown in the diagram

Seconds to ticks

`target = llround(t / time_base) + start_time`. Adding `start_time` means t = 0 is always the first frame, even when the stream starts 10 s into its container.

Frame timestamp

`(best_effort_timestamp − start_time) × time_base`. If the decoder has no best-effort value, `pts` is used instead. A frame with neither is an error. The same rule is used when comparing frames with t, and when reporting the timestamp. This is the frame's own time (0.48), not the requested time (0.5).

Why a seek per sample

Requirement F7 (sample without decoding the whole video). Seeking to the keyframe before each t means the decoder works through at most one keyframe interval per sample.

Color defaults

Requirement F6 (convert color correctly). If a YUV video doesn't specify its color standard, BT.709 is assumed at 720 lines or more and BT.601 below that, both limited range. Values the video does specify are left alone, and so is RGB-coded video.

Identical on every platform

Also F6. The scaler sets `SWS_BITEXACT | SWS_ACCURATE_RND` so x86 and ARM produce the same pixels. `SWS_BITEXACT` alone is not enough: for 8-bit YUV to RGB, swscale uses CPU-specific converters that only `SWS_ACCURATE_RND` turns off. Pixel-hash tests on the fixture clips check this in CI.